

Roteiro — Aula 3: as rotas de autenticação

Deck: `slides/build/aula-03-autenticacao.pptx` (66 slides, sendo 8 de prática) **Apostila da turma:** `03-autenticacao.md` · **Checkpoint:** `aula-03` **Antes de tudo:** `guia-do-instrutor.md` — conduzir a sala, não o conteúdo

Esta é a aula que não cabe. São 1396 linhas de código em 37 arquivos. Digitando, dá 6h30. Leia a seção seguinte antes de qualquer outra coisa.

A decisão que você precisa tomar antes

Você tem duas formas de conduzir esta aula:

Opção A — código pronto, leitura guiada (recomendada)

No começo da aula, todo mundo copia o código do gabarito para **o próprio projeto**:

```
cd ~/capacitacao-gabarito && git checkout aula-03
rsync -a app config db test Gemfile Gemfile.lock ~/automic_auth_api/
cd ~/automic_auth_api && bundle install && bin/rails db:migrate && bin/rails test
```

O `Gemfile` vai junto porque a Aula 3 acrescenta `jwt`, `rack-cors` e `letter_opener` — sem ele a aplicação nem sobe. Os 71 testes verdes são a prova de que a cópia deu certo. **Só depois disso** você percorre os arquivos projetados, explicando decisão por decisão, com eles acompanhando no editor.

A prática vira **modificar** o que já existe, que é o que se faz num emprego de verdade. E o código fica no repositório deles, que é de onde a Aula 4 vai fazer o deploy.

Cabe em ~2h50. É o que este roteiro assume.

Opção B — digitar junto

Só funciona se você tiver 6h ou dividir em dois encontros. Se for por aqui, corte para **três rotas**: cadastro, login e logout. Recuperação de senha vira leitura da apostila.

Antes de começar

- [] O `rsync` do gabarito testado num projeto limpo, com `bin/rails test` verde depois.

- [] Servidor rodando e o Insomnia com as cinco requisições já montadas — você vai fazer o fluxo completo ao vivo duas vezes.
- [] jwt.io aberto numa aba.
- [] Um token válido copiado, pronto para colar no jwt.io.
- [] `tmp/letter_opener/` limpo, para o e-mail da demo ser o primeiro da lista.
- [] Editor com `app/services/` e `app/controllers/api/v1/` já abertos na barra lateral.

Cronograma (Opção A)

As oito práticas são o esqueleto do dia, e nesta aula elas são a aula: o código vem pronto, e o que eles fazem é operar o sistema, quebrar de propósito e entender por quê.

Relógio	Slides	Bloco	Min
00:00	1–5	Abertura, de onde paramos e o combinado de hoje	10
00:10	6–15	Arquitetura: service, serializer, erro, filtro, middleware	26
00:36	16	Prática 1 — traga o código e leia	20
00:56	17–19	Cadastro e enumeração de usuários	10
01:06	20	Prática 2 — cadastro no terminal	20
01:26	—	Intervalo	10
01:36	21–24	Mailer, letter_opener, <code>deliver_now</code>	12
01:48	25	Prática 3 — o e-mail e a confirmação	15
02:03	26–41	Sessão, JWT, login, cookie, XSS, CSRF, 401×403	42
02:45	42	Prática 4 — login, e o token na mão	20
03:05	43–49	Logout: denylist e <code>token_version</code>	22
03:27	50	Prática 5 — logout que revoga de verdade	15
03:42	51–56	Recuperação de senha e a transação	16
03:58	57	Prática 6 — recuperação de senha	20
04:18	58–62	Testes, ferramentas, CORS	14
04:32	63–64	Práticas 7 e 8 — qualidade, e quebre a assinatura	30
05:02	65–66	Recapitulação e fim	4

Dá 5h06 com tudo. É a aula com mais conteúdo conceitual das quatro, e agora com prática de verdade em cada bloco. Corte **nesta ordem**:

#	O que cortar	Ganho
1	Prática 8 (quebrar a assinatura) vira dever de casa, com a apostila	-15
2	Slides 37-38 (XSS e CSRF) — o material fica na apostila	-8
3	Slides 61-62 (ferramentas e CORS) — mostre rodando e siga	-8
4	Slides 21-24 (mailer): mostre só o e-mail abrindo no navegador	-8
5	Prática 1: mande fazer o <code>rsync</code> de casa , na véspera; em aula, só a leitura dos 5 arquivos	-8
6	Prática 2: você faz o cadastro projetado, e eles só montam a coleção do Insomnia	-10
7	Prática 6: faça o segundo <code>confere</code> (enumeração) projetado, e o resto de casa	-10

Cortando de 1 a 4, fecha em **4h07**. Cortando os sete, **3h19**.

Nunca corte as Práticas 4 e 5. O token na mão e o logout que revoga são o ponto alto da aula, e são o que diferencia esta capacitação de um tutorial de YouTube.

Bloco a bloco

00:00 — Abertura e o combinado (1-5)

O slide 3 é o mapa da aula inteira. Deixe ele na tela enquanto fala a regra:

Controller recebe requisição e devolve resposta. Regra de negócio não mora nele.

00:10 — Arquitetura (6-15)

Abra `app/services/users/register.rb` projetado e conte as seis coisas que ele faz. Depois pergunte: "quanto disso vocês conseguiriam testar sem subir uma requisição HTTP inteira?"

- **7** — serializer. Faça a demonstração do susto: "o que acontece se eu escrever `render json: user`?"
Mostre no console:

```
ruby User.first.as_json.keys
```

Está lá o `password_digest` e os digests dos códigos. **Uma linha, e vazou.**

- **10-12** — `before_action` e middleware. O slide 12 é o mapa completo. Se der, rode:

```
bash bin/rails middleware
```

- **13** — strong parameters. A frase: "sem isso, alguém manda `role: admin` no cadastro e vira admin. Isso tem nome: mass assignment, e já derrubou sistema grande."

00:56 — Cadastro (17–19)

O slide 19 é o primeiro dos quatro momentos de "enumeração" da aula. Marque isso: você vai voltar nele três vezes, e no fim eles devem conseguir prever a decisão sozinhos.

Pergunta para a turma antes de virar o slide: *"o e-mail já existe. O que a API deve responder?"* Alguém vai dizer "este e-mail já está cadastrado". Aí você mostra por que não.

01:36 — E-mail (21–24)

Faça o cadastro ao vivo no Insomnia e **mostre o e-mail abrindo no navegador** pelo `letter_opener`. É um daqueles momentos em que a turma acorda.

No slide 24, a decisão contra o livro-texto: `deliver_now` porque, numa fila, o código de 6 dígitos ficaria gravado **em texto** na tabela de jobs.

02:03 — JWT (26–33)

O bloco conceitual mais denso. Comece pelo problema (27): *"lembram que HTTP não tem memória? Então como o servidor sabe que vocês já entraram?"*

Faça a demo do jwt.io. Cole o token que você preparou e mostre o payload legível na tela.

"Está assinado. Não está criptografado. Qualquer um lê. Então nunca ponham aqui nada que não possa ser lido."

No 33, o `true` do `JWT.decode`. Diga que já foi CVE em várias bibliotecas, e que eles vão brincar com isso na prática.

02:24 — Login (34–41)

- **35** — os dois transportes. O `client` no corpo existe por isso.
- **36–37** — o que é cookie, e as três flags.
- **38–39** — XSS e CSRF. Se o tempo apertar, é aqui que você corta.
- **41** — o segundo "enumeração": mesma mensagem para e-mail inexistente e senha errada. E o `DUMMY_PASSWORD_DIGEST` da Aula 2 volta, fechando o buraco pelo lado do tempo.

03:05 — Logout (43–49)

O ponto alto. Conduza como um problema, não como uma solução.

1. Pergunte: *"o token vale 24h e o servidor não guarda sessão. O que 'sair' significa?"*
2. Deixe a turma propor. Alguém vai dizer "apaga no cliente". Aceite e pergunte: *"e se alguém já copiou o token?"*
3. Aí você apresenta a denylist (46).
4. O slide 47 é a sacada: o TTL de cada entrada é o que faltava para o token expirar, então **a lista se limpa sozinha**.

5. E o 48 mostra a outra ferramenta: `token_version` derruba tudo de uma vez.

Demonstre ao vivo. Vale mais que os seis slides:

```
TOKEN=... # pegue do login
curl $API/me -H "Authorization: Bearer $TOKEN" # 200
curl -X DELETE $API/sessions -H "Authorization: Bearer $TOKEN"
curl -i $API/me -H "Authorization: Bearer $TOKEN" # 401
```

03:42 — Recuperação de senha (51–56)

Terceiro e quarto "enumeração" (53). A essa altura, **pergunte antes**: *"o e-mail não existe. O que respondemos?"* Eles devem acertar sozinhos.

O slide 55 amarra com a Aula 2: a transação. E o 56 tem o ponto do fluxo inteiro —

`invalidate_sessions!`: *"se alguém invadiu a conta, trocar a senha tem que expulsar o invasor. Sem essa linha, ele continua logado."*

Demonstre: faça o reset e mostre o token antigo virando 401.

04:18 — Testes, ferramentas e CORS (58–62)

Rápido, e é o que amarra com a Aula 4.

- **59** — o teste que resume a aula: login → logout → `/me` dá 401. Projete-o.
- **60** — a pegadinha de teste: `assert_response :unauthorized`, e não `assert_equal 401`.
- **61** — RuboCop, Brakeman, bundler-audit. Diga que os três viram o **portão** do CI na Aula 4.
- **62** — CORS. A frase: *"o navegador é quem bloqueia, não o servidor. Por isso `curl` funciona e a página não."*

Conduzindo as oito práticas

Nesta aula a prática **é** a aula: o código vem pronto, e o trabalho é operar, quebrar e entender. Cada uma tem, na apostila, os comandos, a conferência e uma tabela de erros.

#	Slide	O que cobrar em voz alta
1	16	71 testes verdes antes de seguir. Sem isso, nada depois funciona
2	20	O <code>Content-Type</code> esquecido é o erro nº 1. E a mensagem do e-mail repetido: entrega quem tem conta?
3	25	Login antes de confirmar dá 403 , não 401. A senha estava certa
4	42	O servidor não guardou sessão nenhuma. Ele leu o token
5	50	O <code>401</code> depois do logout com um token ainda válido. É a prática que justifica a aula
6	57	O e-mail que não existe responde igual . Faça lado a lado, na tela
7	63	Quebrar a denylist e ver qual teste acusa
8	64	Desfazer o <code>false</code> do <code>JWT.decode</code> . Ninguém sai da sala com essa linha

Dois cuidados de condução:

Na Prática 4, se o `$TOKEN` sair vazio para alguém, mande rodar o `curl -i` sozinho e olhar se o cabeçalho `Authorization` está lá. É quase sempre aspas quebradas no shell — e a saída é o Insomnia.

Na Prática 8, cobre o desfazer em voz alta:

```
git checkout app/services/auth/decode_token.rb
bin/rails test
```

É uma falha de autenticação real. Não deixe ninguém ir embora com ela no disco.

Perguntas que vão aparecer

Pergunta	Resposta curta
"Se o payload é público, não é inseguro?"	Não, desde que você não coloque segredo lá. O token prova quem você é; ele não precisa esconder isso de você.
"Por que não usar sessão como todo mundo?"	Porque o cliente principal é app nativo, que não tem cookie. E porque assim qualquer servidor valida sozinho, sem sessão compartilhada.
"Então JWT é pior que sessão?"	É diferente. Sessão facilita o logout e complica o escalar; token faz o contrário. Nós resolvemos o logout com a denylist.
"A denylist não é uma sessão disfarçada?"	Um pouco — mas só guarda os tokens revogados, não todos, e expira sozinha. É bem mais barata.
"Por que 15 minutos no código, e não 1 hora?"	Janela curta reduz o tempo em que um código interceptado serve. 15 min é o suficiente para checar e-mail sem correr.
"E se o e-mail do usuário estiver errado no cadastro?"	Ele nunca confirma, nunca loga, e a conta fica órfã. É por isso que existe o <code>resend</code> .
"Posso usar Devise em vez de escrever tudo isso?"	Pode, e em projeto de verdade normalmente é o que se faz. Mas você não entenderia nada do que ele faz — e é isso que estamos aprendendo.
"Por que <code>unprocessable_content</code> e não <code>unprocessable_entity</code> ?"	Mesmo código 422. O nome foi renomeado na especificação, e o Rails 8 seguiu.

Dever de casa

1. Fazer os dois bônus.
2. Ler `app/services/users/complete_password_reset.rb` inteiro e explicar, por escrito, por que a política de senha é validada **antes** de consumir o código.
3. Rodar `bin/brakeman` e ler o que ele procura.